

```

"""
test_dh_ds_convergence.py
=====

Mesure d_h (dimension de Hausdorff via boules géodésiques)
et d_s (dimension spectrale via comptage d'eigenvalues du Laplacien)
sur l'équilibre MCMC du Blob, pour N ∈ {200, 500, 1000, 2000}.

But : déterminer si l'écart d_h ≈ 1.77 vs d_s ≈ 3.05 observé à N=200
      est un effet de taille finie (d_h → 3 à grand N) ou une signature
      géométrique persistante (Blob fractal/hyperbolique).

Optimisations critiques :
- Comptage incrémental des triangles (O(N) par pas MCMC au lieu de O(N³))
- Liste d'arêtes maintenue (O(|E|) au lieu de O(N²))
- Mise à jour incrémentale du vecteur de degrés
"""

import numpy as np
from scipy.spatial import distance_matrix
from scipy.special import gamma as gamma_func
from scipy.stats import linregress
from scipy.linalg import eigvalsh
import time
import json
from collections import deque

KAPPA0 = 8
BETA = 0.30
TAU = 0.50

# ===== Helpers Blob =====
def rips_radius(N, d, kappa0=KAPPA0):
    Vd = np.pi**(d/2) / gamma_func(d/2 + 1)
    return (kappa0 / (N * Vd))**(1/d)

def build_rips(N, d, seed):
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0, 1, (N, d))
    r = rips_radius(N, d)
    dist = distance_matrix(pts, pts)
    adj = ((dist < r) & (dist > 0)).astype(np.int8)
    return pts, adj, r, dist

def count_triangles_full(adj):

```

```

A = adj.astype(np.int32)
A3 = A @ A @ A
return int(np.trace(A3)) // 6

# ===== MCMC optimisé =====
def run_mcmc_fast(N, d, seed, n_steps, verbose=False):
    """MCMC V109 avec comptage incrémental. O(|E|+N) par pas."""
    pts, adj, r, dist = build_rips(N, d, seed)
    pool = ((dist < 1.5 * r) & (dist > 0)).astype(np.int8)
    rng = np.random.default_rng(seed + 9991)

    # État maintenu incrémentalement
    n_tri = count_triangles_full(adj)
    deg = adj.sum(axis=1).astype(np.int64)
    # Liste d'arêtes
    upper = np.argwhere(np.triu(adj, 1))
    edges_list = [(int(u), int(v)) for u, v in upper]

    def current_S(n_tri_loc, deg_loc):
        mu = deg_loc.mean()
        if mu < 1e-9:
            return -1e9
        D = deg_loc.std() / mu
        return float(np.log(1 + n_tri_loc) - BETA * D)

    S_cur = current_S(n_tri, deg)
    n_accept = 0

    for step in range(n_steps):
        if not edges_list:
            break
        # Pick edge to remove
        e_idx = rng.integers(len(edges_list))
        u_rm, v_rm = edges_list[e_idx]
        # Pick edge to add (in pool, not already present)
        u_add = v_add = -1
        for _ in range(100):
            i = int(rng.integers(N))
            j = int(rng.integers(N))
            if i != j and pool[i, j] == 1 and adj[i, j] == 0:
                u_add, v_add = (i, j) if i < j else (j, i)
                break
        if u_add < 0:
            continue

        # Comptage incrémental
        common_rm = int((adj[u_rm] & adj[v_rm]).sum())

```

```

common_add = int((adj[u_add] & adj[v_add]).sum())
n_tri_new = n_tri - common_rm + common_add

# Mise à jour temporaire degrés
deg_new = deg.copy()
deg_new[u_rm] -= 1
deg_new[v_rm] -= 1
deg_new[u_add] += 1
deg_new[v_add] += 1

S_new = current_S(n_tri_new, deg_new)
delta = (S_new - S_cur) * N / TAU

if delta > 0 or rng.random() < np.exp(min(0.0, delta)):
    # Accept
    adj[u_rm, v_rm] = adj[v_rm, u_rm] = 0
    adj[u_add, v_add] = adj[v_add, u_add] = 1
    edges_list[e_idx] = (u_add, v_add)
    n_tri = n_tri_new
    deg = deg_new
    S_cur = S_new
    n_accept += 1

    if verbose and (step + 1) % max(1, n_steps // 5) == 0:
        print(f"      [step {step+1}/{n_steps}] n_tri={n_tri}, accept={n_accept}
              f"({100*n_accept/(step+1):.1f}%)"")

return adj, n_tri, n_accept

# ===== d_h via BFS ball volumes =====
def bfs_distances(adj, source, max_radius):
    N = adj.shape[0]
    dist = -np.ones(N, dtype=int)
    dist[source] = 0
    frontier = deque([source])
    while frontier:
        v = frontier.popleft()
        if dist[v] >= max_radius:
            continue
        neighbors = np.where(adj[v] == 1)[0]
        for u in neighbors:
            if dist[u] == -1:
                dist[u] = dist[v] + 1
                frontier.append(u)
    return dist

def measure_d_h(adj, n_centers, max_radius, rng):

```

```

N = adj.shape[0]
centers = rng.choice(N, min(n_centers, N), replace=False)
slopes = []
for c in centers:
    distv = bfs_distances(adj, int(c), max_radius)
    rs, vols = [], []
    for r in range(1, max_radius + 1):
        vol = int(((distv >= 0) & (distv <= r)).sum())
        rs.append(r)
        vols.append(vol)
    # Filtre : volumes > 1 et r >= 2 (évite artefacts de bord)
    valid = [(r, v) for r, v in zip(rs, vols) if v > 1 and r >= 2 and v < N]
    if len(valid) < 3:
        continue
    rs_arr = np.array([r for r, _ in valid], dtype=float)
    vs_arr = np.array([v for _, v in valid], dtype=float)
    res = linregress(np.log(rs_arr), np.log(vs_arr))
    slopes.append(res.slope)
if len(slopes) < 3:
    return None, None
return float(np.mean(slopes)), float(np.std(slopes))

# ===== d_s via Laplacien =====
def measure_d_s(adj):
    N = adj.shape[0]
    deg = adj.sum(axis=1).astype(np.float64)
    L = np.diag(deg) - adj.astype(np.float64)
    eigvals = eigvalsh(L)
    eigvals = np.sort(eigvals)
    # Drop trivial zeros
    eigvals = eigvals[eigvals > 1e-9]
    if len(eigvals) < 20:
        return None, None
    # Counting function  $N(\lambda) \sim \lambda^{\{d_s/2\}}$ 
    lam_max = eigvals.max()
    # Use lower portion of spectrum:  $\lambda \in [\lambda_{\min}, 0.15 * \lambda_{\max}]$ 
    threshold = 0.15 * lam_max
    mask = eigvals <= threshold
    n_low = int(mask.sum())
    if n_low < 10:
        n_low = max(20, len(eigvals) // 4)
    lam_arr = eigvals[:n_low]
    N_arr = np.arange(1, n_low + 1)
    res = linregress(np.log(lam_arr), np.log(N_arr))
    d_s = 2 * res.slope
    return float(d_s), float(res.rvalue**2)

```

```

# ===== Main =====
if __name__ == "__main__":
    Ns = [200, 500, 1000, 2000]
    SEEDS = [0, 1, 2, 3, 4]

    print(f"=== Test convergence d_h vs d_s ===")
    print(f"N = {Ns}, seeds = {SEEDS}")
    print(f"Paramètres V109: kappa0={KAPPA0}, beta={BETA}, tau={TAU}\n")

    all_results = {}
    t_global = time.time()

    for N in Ns:
        n_steps = max(1500, 6 * N) # équilibration linéaire en N
        max_r = max(5, int(round(N**(1/3) * 1.8)))
        n_centers = min(40, N // 5)

        print(f"\n{'='*60}")
        print(f"N = {N} (n_steps={n_steps}, max_r={max_r}, n_centers={n_centers})")
        print(f"{'='*60}")

        seed_data = []
        for seed in SEEDS:
            t0 = time.time()
            adj, n_tri, n_acc = run_mcmc_fast(N, 3, seed, n_steps)
            t_mcmc = time.time() - t0

            t1 = time.time()
            rng_h = np.random.default_rng(seed + 70000)
            d_h, d_h_std = measure_d_h(adj, n_centers, max_r, rng_h)
            t_dh = time.time() - t1

            t2 = time.time()
            d_s, d_s_r2 = measure_d_s(adj)
            t_ds = time.time() - t2

            seed_data.append({
                'seed': seed, 'n_tri': int(n_tri), 'n_accept': n_acc,
                'd_h': d_h, 'd_h_std': d_h_std,
                'd_s': d_s, 'd_s_r2': d_s_r2,
                't_mcmc': t_mcmc, 't_dh': t_dh, 't_ds': t_ds,
            })

        print(f" seed {seed}: d_h={d_h:.3f}±{d_h_std:.3f} d_s={d_s:.3f} (R² "
              f"n_tri={n_tri} accept={100*n_acc/n_steps:.0f}% "
              f"[mcmc {t_mcmc:.1f}s + dh {t_dh:.1f}s + ds {t_ds:.1f}s]")

    dh_vals = [r['d_h'] for r in seed_data if r['d_h'] is not None]

```

```

ds_vals = [r['d_s'] for r in seed_data if r['d_s'] is not None]
ntri_vals = [r['n_tri'] for r in seed_data]

all_results[N] = {
    'd_h_mean': float(np.mean(dh_vals)),
    'd_h_sem': float(np.std(dh_vals) / np.sqrt(len(dh_vals))),
    'd_h_std': float(np.std(dh_vals)),
    'd_s_mean': float(np.mean(ds_vals)),
    'd_s_sem': float(np.std(ds_vals) / np.sqrt(len(ds_vals))),
    'd_s_std': float(np.std(ds_vals)),
    'n_tri_mean': float(np.mean(ntri_vals)),
    'n_tri_std': float(np.std(ntri_vals)),
    'per_seed': seed_data,
}

r = all_results[N]
print(f"\n ⇒ N={N}: d_h = {r['d_h_mean']:.3f} ± {r['d_h_std']:.3f} | "
      f"d_s = {r['d_s_mean']:.3f} ± {r['d_s_std']:.3f} | "
      f"écart Δ = {r['d_s_mean'] - r['d_h_mean']:+.3f}")

# === RÉSUMÉ ===
print(f"\n\n{'='*70}")
print("RÉSUMÉ FINAL – convergence des dimensions")
print(f"{'='*70}")
print(f"{'N':>8} {'d_h (Hausdorff)':>20} {'d_s (spectral)':>20} {'Δ = d_s"}
print("-" * 70)
for N in Ns:
    r = all_results[N]
    print(f"{'N':>8} {r['d_h_mean']:>6.3f} ± {r['d_h_std']:.3f} "
          f"{r['d_s_mean']:>6.3f} ± {r['d_s_std']:.3f} "
          f"{r['d_s_mean'] - r['d_h_mean']:>+6.3f}")

# Trend analysis
print(f"\n--- Tendance d_h en fonction de N ---")
dh_means = [all_results[N]['d_h_mean'] for N in Ns]
ds_means = [all_results[N]['d_s_mean'] for N in Ns]
print(f"d_h : {dh_means[0]:.3f} → {dh_means[-1]:.3f} "
      f"Δ = {dh_means[-1] - dh_means[0]:+.3f} de N={Ns[0]} à N={Ns[-1]}")
print(f"d_s : {ds_means[0]:.3f} → {ds_means[-1]:.3f} "
      f"Δ = {ds_means[-1] - ds_means[0]:+.3f}")

# Verdict
print(f"\n--- VERDICT ---")
if abs(dh_means[-1] - 3.0) < 0.3:
    print(f"d_h(N={Ns[-1]}) = {dh_means[-1]:.3f} ≈ 3 : géométrie 3D-euclidien")
    print(f"⇒ écart d_h vs d_s à petit N était un effet de taille finie.")
    print(f"⇒ Test loi d'aire (Bekenstein-Hawking) peut être relancé propre")
elif dh_means[-1] < dh_means[0] - 0.1:

```

```

    print(f"d_h DÉCROÎT avec N : {dh_means[0]:.3f} → {dh_means[-1]:.3f}")
    print(f"    ⇒ géométrie de plus en plus fractale/hyperbolique à grande échelle")
elif abs(dh_means[-1] - dh_means[0]) < 0.2:
    print(f"d_h STABLE à ~{dh_means[-1]:.2f}, persistant à grand N.")
    print(f"    ⇒ signature géométrique réelle : Blob est fractal en Hausdorff")
    print(f"        mais diffuse en 3D spectrale. Reformulation théorique nécessaire")
else:
    print(f"d_h évolue vers {dh_means[-1]:.3f}, à interpréter avec N plus grand")

t_total = time.time() - t_global
print(f"\nTemps total : {t_total:.1f}s")

with open('/home/claude/dh_ds_convergence.json', 'w') as f:
    json.dump({'Ns': Ns, 'seeds': SEEDS, 'results': {str(k): v for k, v in al.items()},
              'wall_time_s': t_total}, f, indent=2, default=str)
print("Sauvegarde : /home/claude/dh_ds_convergence.json")

```